

Index & General Template

INDEX

Section	Page
Index and General Templ	1
Range Query DSA	2
Graph Algorithms	4
Data Structures	7
String Algorithms	10
Math and Number Theory	13
Dynamic Programming	17
Greedy and Sortings	19

GENERAL TEMPLATE

```
#include <bits/stdc++.h>

using namespace std;

#define int long long

#define M 1000000007

#define N 1000000

#define INF 1e17

#define endl "\n"

#define all(v) v.begin(), v.end()

#define yes cout<<"Yes"<<endl

#define no cout<<"No"<<endl

#define minus cout<<"-1"<<endl

#define make_unique(x) sort(all(x)); \
x.erase(unique(all(x)),x.end())

void marwan(){}

int32_t main()

{

ios_base::sync_with_stdio(false); cin.tie(0);

    int t;

    cin>>t;

    while(t--) marwan();

    return 0;

}
```

=====

Topic-01: Range Query

=====

Prefix Sum

```
// Build O(N), Query O(1)
vector<int>pre(n+1);
for(int i=1;i<=n;i++)pre[i]=pre[i-1]+a[i];
int query(int l,int r){return pre[r]-pre[l-1];}
```

Prefix XOR

```
// Build O(N), Query O(1)
vector<int>pre(n+1);
for(int i=1;i<=n;i++)pre[i]=pre[i-1]^a[i];
int query(int l,int r){return pre[r]^pre[l-1];}
```

Difference Array

```
// Range Update O(1)
vector<int>diff(n+2);
diff[l]+=x;
diff[r+1]-=x;
for(int i=1;i<=n;i++)diff[i]+=diff[i-1];
```

2D Prefix Sum

```
vector<vector<int>>>pre(n+1,vector<int>(m+1));
for(int i=1;i<=n;i++)
for(int j=1;j<=m;j++)
pre[i][j]=a[i][j]+pre[i-1][j]+pre[i][j-1]-pre[i-1][j-1];
int query(int x1,int y1,int x2,int y2){
return pre[x2][y2]-pre[x1-1][y2]-pre[x2][y1-1]+pre[x1-1][y1-1];
}
```

Fenwick Tree (BIT)

```
struct BIT{
int n;vector<int>bit;
```

```
BIT(int n):n(n){bit.assign(n+1,0);}
void add(int i,int v){for(;i<=n;i+=i&-i)bit[i]+=v;}
int sum(int i){int s=0;for(;i-=i&-i)s+=bit[i];return s;}
int query(int l,int r){return sum(r)-sum(l-1);}
};
```

XOR Fenwick

```
struct BIT{
int n;vector<int>bit;
BIT(int n):n(n){bit.assign(n+1,0);}
void add(int i,int x){for(;i<=n;i+=i&-i)bit[i]^=x;}
int sum(int i){int s=0;for(;i-=i&-i)s^=bit[i];return s;}
int query(int l,int r){return sum(r)^sum(l-1);}
};
```

Segment Tree

```
class SGTree{
public:
vector<int>seg;
SGTree(int n){seg.resize(4*n+1);}
void build(int i,int l,int r,vector<int>&a){
if(l==r){seg[i]=a[l];return;}
int m=(l+r)>>1;
build(i*2+1,l,m,a);
build(i*2+2,m+1,r,a);
seg[i]=seg[i*2+1]+seg[i*2+2];
}
void update(int i,int l,int r,int p,int v){
if(l==r){seg[i]=v;return;}
int m=(l+r)>>1;
if(p<=m)update(i*2+1,l,m,p,v);
else update(i*2+2,m+1,r,p,v);
seg[i]=seg[i*2+1]+seg[i*2+2];
}
int query(int i,int l,int r,int ql,int qr){
if(qr<l || r<ql)return 0;
if(ql<=l&&r<=qr)return seg[i];
int m=(l+r)>>1;
return
```

```

query(i*2+1,l,m,ql,q,r)+query(i*2+2,m+1,r,ql,q,r);
}
};

```

Usage

```

SGTree st(n);
st.build(0,0,n-1,a);
st.query(0,0,n-1,l,r);
st.update(0,0,n-1,pos,val);

```

Lazy Segment Tree

```

vector<int>seg(4*N),lazy(4*N);
void push(int i,int l,int r){
if(!lazy[i])return;
seg[i]+=(r-l+1)*lazy[i];
if(l!=r){
lazy[i<<1]+=lazy[i];
lazy[i<<1|1]+=lazy[i];
}
lazy[i]=0;
}
void update(int i,int l,int r,int ql,int qr,int v){
push(i,l,r);
if(qr<l||r<ql)return;
if(ql<=l&&r<=qr){lazy[i]+=v;push(i,l,r);return;}
int m=(l+r)>>1;
update(i<<1,l,m,ql,q,r,v);
update(i<<1|1,m+1,r,ql,q,r,v);
seg[i]=seg[i<<1]+seg[i<<1|1];
}
int query(int i,int l,int r,int ql,int qr){
push(i,l,r);
if(qr<l||r<ql)return 0;
if(ql<=l&&r<=qr)return seg[i];
int m=(l+r)>>1;
return
query(i<<1,l,m,ql,q,r)+query(i<<1|1,m+1,r,ql,q,r);
}

```

Sparse Table

```

const int LG=20;
vector<vector<int>>st(LG,vector<int>(N));
for(int i=0;i<n;i++)st[0][i]=a[i];
for(int j=1;j<LG;j++)
for(int i=0;i+(1<<j)<=n;i++)
st[j][i]=min(st[j-1][i],st[j-1][i+(1<<(j-1))]);
int query(int l,int r){
int k=31-__builtin_clz(r-l+1);
return min(st[k][l],st[k][r-(1<<k)+1]); }

```

Mo's Algorithm

```

const int B=450;
struct Query{
int l,r,id;
bool operator<(Query o)const{
if(l/B!=o.l/B)return l<o.l;
return (l/B)&1?r>o.r:r<o.r; }
};
sort(all(q));
int L=0,R=-1;
for(auto x:q){
while(L>x.l)add(--L);
while(R<x.r)add(++R);
while(L<x.l)remove(L--);
while(R>x.r)remove(R--);
ans[x.id]=cur;
}

```

Complexity

DS	Build	Update	Query
Prefix Sum	O(N)	—	O(1)
Prefix XOR	O(N)	—	O(1)
Difference	O(N)	O(1)	O(N) rebuild
Fenwick	O(N)	O(logN)	O(logN)
Segment Tree	O(N)	O(logN)	O(logN)
Lazy SegTree	O(N)	O(logN)	O(logN)

=====

Graph

=====

Graph Input (Unweighted)

```
int n;
cin>>n;
vector<vector<int>>>g(n+1);
vector<int>vis(n+1);
for(int i=0;i+1<n;i++){
    int u,v;
    cin>>u>>v;
    g[u].push_back(v);
    g[v].push_back(u);
}
```

Weighted Graph

```
int n,m;
cin>>n>>m;
vector<vector<pair<int,int>>>g(n+1);
for(int i=0;i<m;i++){
    int u,v,w;
    cin>>u>>v>>w;
    g[u].push_back({v,w});
    g[v].push_back({u,w});
}
```

DFS

```
void dfs(int
u,vector<vector<int>>>&g,vector<int>&vis){
    vis[u]=1;
    for(auto v:g[u])
        if(!vis[v])dfs(v,g,vis);
}
```

BFS

```
vector<int>d(n+1,-1);
queue<int>q;
q.push(s);
d[s]=0;
while(!q.empty()){
    int u=q.front();q.pop();
    for(auto v:g[u])
        if(d[v]==-1){
            d[v]=d[u]+1;
            q.push(v);
        }
}
```

Multi Source BFS

```
queue<int>q;
for(auto s:src){
    q.push(s);
    d[s]=0;
}
while(!q.empty()){
    int u=q.front();q.pop();
    for(auto v:g[u])
        if(d[v]==-1){
            d[v]=d[u]+1;
            q.push(v);
        }
}
```

0-1 BFS

```
deque<int>dq;
vector<int>d(n+1,INF);
dq.push_front(s);
d[s]=0;
while(!dq.empty()){
    int u=dq.front();
    dq.pop_front();
    for(auto [v,w]:g[u]){
        if(d[v]>d[u]+w){
            d[v]=d[u]+w;

```

```

        if(w)dq.push_back(v);
        else dq.push_front(v);
    }
}

```

Dijkstra

```

vector<int>d(n+1,INF);
priority_queue<pair<int,int>,vector<pair<int,int>>,
greater<pair<int,int>>>pq;
pq.push({0,s});
d[s]=0;
while(!pq.empty()){
    auto [dis,u]=pq.top();pq.pop();
    if(dis>d[u])continue;
    for(auto [v,w]:g[u]){
        if(d[v]>dis+w){
            d[v]=dis+w;
            pq.push({d[v],v});
        }
    }
}

```

Bellman Ford

```

vector<int>d(n+1,INF);
d[s]=0;
for(int i=1;i<n;i++)
for(auto [u,v,w]:edge)
if(d[u]!=INF&& d[v]>d[u]+w)
d[v]=d[u]+w;

```

Negative Cycle

```

bool neg=0;
for(auto [u,v,w]:edge)
if(d[u]!=INF&& d[v]>d[u]+w)
neg=1;

```

Floyd Warshall

```

for(int k=1;k<=n;k++)
for(int i=1;i<=n;i++)
for(int j=1;j<=n;j++)
d[i][j]=min(d[i][j],d[i][k]+d[k][j]);

```

Topological Sort (Kahn)

```

queue<int>q;
for(int i=1;i<=n;i++)
if(!in[i])q.push(i);
while(!q.empty()){
    int u=q.front();q.pop();
    topo.push_back(u);
    for(auto v:g[u])
        if(--in[v]==0)
            q.push(v);
}

```

Cycle Detection (Undirected)

```

bool dfs(int u,int p){
    vis[u]=1;
    for(auto v:g[u]){
        if(v==p)continue;
        if(vis[v])return 1;
        if(dfs(v,u))return 1;
    }
    return 0;
}

```

Cycle Detection (Directed)

```

bool dfs(int u){
    vis[u]=1;
    for(auto v:g[u]){
        if(vis[v]==1)return 1;
        if(!vis[v]&&dfs(v))return 1;
    }
    vis[u]=2;
    return 0;
}

```

Bipartite

```
void dfs(int u, int c){
    vis[u]=1; color[u] = c;
    for(auto v:g[u]){

if(vis[v] && color[v] == c) flag = false;
        if(!vis[v]) dfs(v,);
    }
}
```

Connected Components

```
int cc=0;
for(int i=1;i<=n;i++){
    if(!vis[i]){
        cc++;
        dfs(i,g,vis);
    }
}
```

DSU

```
struct DSU {
    vector<int> p, sz;

    DSU(int n) : p(n+1), sz(n+1,1) {
        iota(p.begin(), p.end(), 0);
    }

    int find(int x) {
        return p[x] == x ? x : p[x] = find(p[x]);
    }

    void unite(int a, int b) {
        a = find(a);
        b = find(b);

        if (a != b) {
            p[b] = a;
```

```
sz[a] += sz[b];
```

```
}
```

```
}
```

```
};
```

Kruskal

```
sort(all(edge));
DSU dsu(n);
int mst=0;
for(auto [w,u,v]:edge)
    if(dsu.find(u)!=dsu.find(v)){
        mst+=w;
        dsu.unite(u,v);
    }
```

Prim

```
priority_queue<pair<int,int>,vector<pair<int,int>>,
greater<pair<int,int>>>pq;
pq.push({0,1});
while(!pq.empty()){
    auto [w,u]=pq.top();pq.pop();
    if(vis[u])continue;
    vis[u]=1;
    mst+=w;
    for(auto [v,c]:g[u])
        if(!vis[v])pq.push({c,v});
}
```

Tree Diameter (2 BFS)

```
BFS(1)->u;
BFS(u)->v;
dist[v]=diameter;
```

DATA STRUCTURES

PBDS (Ordered Set)

This should match your snippet.

```
#include<ext/pb_ds/assoc_container.hpp>
#include<ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
```

```
template<class T>
using ordered_set=
tree<T,null_type,less<T>,rb_tree_tag,
tree_order_statistics_node_update>;
```

Operations

```
ordered_set<int>st;
```

```
st.insert(x);
st.erase(x);
```

```
st.find_by_order(k);
```

```
st.order_of_key(x);
```

PBDS (Duplicates)

```
template<class T>
using ordered_multiset=
tree<T,null_type,less_equal<T>,
rb_tree_tag, tree_order_statistics_node_update>;
```

Trie

```
struct Node{
Node* nxt[26];
bool end=0;
Node(){memset(nxt,0,sizeof(nxt));}
};
```

```
Node*root=new Node();
```

```
void insert(string s){
Node*cur=root;
for(char c:s){
int x=c-'a';
if(!cur->nxt[x])cur->nxt[x]=new Node();
cur=cur->nxt[x];
}
cur->end=1;
}
```

```
bool search(string s){
Node*cur=root;
for(char c:s){
int x=c-'a';
if(!cur->nxt[x])return 0;
cur=cur->nxt[x];
}
return cur->end;
}
```

Binary Trie (Maximum XOR)

```
struct Node{
Node*bit[2]={};
};
```

```
void insert(int x){
Node*cur=root;
for(int i=31;i>=0;i--){
int b=(x>>i)&1;
if(!cur->bit[b])cur->bit[b]=new Node();
cur=cur->bit[b];
}
}
```

```
int query(int x){
Node*cur=root;
int ans=0;
for(int i=31;i>=0;i--){
int b=(x>>i)&1;
if(cur->bit[b^1]){
```

```
ans|=1<<i;
cur=cur->bit[b^1]; }
else cur=cur->bit[b]; }
return ans; }
```

Monotonic Stack

Nearest Smaller Left

```
stack<int>st;
for(int i=0;i<n;i++){
while(!st.empty()&&st.top()>a[i])
st.pop();
ans[i]=st.empty()? -1:st.top();
st.push(a[i]);
}
```

Monotonic Stack (Index)

```
stack<int>st;
for(int i=0;i<n;i++){
while(!st.empty()&&a[st.top()]>=a[i])
st.pop();
L[i]=st.empty()? -1:st.top();
st.push(i);
}
```

Monotonic Queue

Sliding Maximum

```
deque<int>dq;
for(int i=0;i<n;i++){
while(!dq.empty()&&dq.front()<=i-k)
dq.pop_front();
while(!dq.empty()&&a[dq.back()]<=a[i])
dq.pop_back();
dq.push_back(i);
if(i>=k-1)
cout<<a[dq.front()]<<" ";
}
```

Priority Queue

Max Heap

```
priority_queue<int>pq;
```

Min Heap

```
priority_queue<int,vector<int>,greater<int>>pq;
```

Pair Heap

```
priority_queue<pair<int,int>,
vector<pair<int,int>>,
greater<pair<int,int>>>pq;
```

Deque

```
deque<int>dq;
dq.push_front(x);
dq.push_back(x);
dq.pop_front();
dq.pop_back();
dq.front();
dq.back();
```

Stack

```
stack<int>st;
st.push(x);
st.pop();
st.top();
st.empty();
```

Queue

```
queue<int>q;
q.push(x);
q.pop();

q.front();
q.empty();
```

Coordinate Compression

```
vector<int>c=v;
sort(all(c));
c.erase(unique(all(c)),c.end());
for(auto &x:v)
x=lower_bound(all(c),x)-c.begin();
```

Heap Tricks

Largest K

```
priority_queue<int,vector<int>,greater<int>>pq;
for(auto x:v){
pq.push(x);
if(pq.size()>k)pq.pop();
}
```

Median

```
priority_queue<int>L;
priority_queue<int,vector<int>,greater<int>>R;
```

STL Containers

```
vector<int>v;

set<int>s;

multiset<int>ms;

map<int,int>mp;

unordered_map<int,int>ump;

unordered_set<int>us;

stack<int>st;

queue<int>q;

deque<int>dq;

priority_queue<int>pq;
```

Pair / Tuple

```
auto [x,y]=p;
auto [a,b,c]=t;
```

Container Erase While Iterating

```
for(auto it=s.begin();it!=s.end();){
    if(*it==x)it=s.erase(it);
    else ++it;
}
```

stringstream

```
stringstream ss(s);
string x;
while(ss>>x);
```

Random

```
mt19937 rng(
chrono::steady_clock::
now().time_since_epoch().count());
shuffle(all(v),rng);
```

String

```
s.substr(l,len);
s.find(t);
s.rfind(t);
s.find_first_of("aeiou");
s.find_last_of('/');
s.erase(pos,len);
s.insert(pos,t);
s.replace(pos,len,t);
reverse(all(s));
stoi(s);
stoll(s);
to_string(x);
```

=====

STRING ALGORITHMS

=====

String Basics

Use: Frequently used STL operations.

Note:

Most string problems require these operations before advanced algorithms.

```
string s;
cin>>s;
int n=s.size();    // length
reverse(all(s));   // reverse string
sort(all(s));      // sort characters
```

Prefix Function (KMP)

Complexity: $O(N)$

Use: Find **all occurrences** of a pattern in a text.

Idea: Build the **LPS (Longest Prefix Suffix)** array. Whenever mismatch occurs, jump using LPS instead of starting over.

```
vector<int>prefix(string s){
int n=s.size();
vector<int>pi(n);
for(int i=1,j=0;i<n;){
if(s[i]==s[j])pi[i++]=++j;
else if(j)j=pi[j-1];
else i++;
}
return pi;
}
```

KMP Search

Use: Search pattern inside text.

Idea: Reuse previously matched prefix using the LPS array.

```
vector<int>pi=prefix(p);
for(int i=0,j=0;i<n;){
if(s[i]==p[j])i++,j++;
else if(j)j=pi[j-1];
else i++;
if(j==m){
cout<<i-m<<endl;
j=pi[j-1];
}
}
```

Z Algorithm

Complexity: $O(N)$

Use: Prefix matching, border problems, string periodicity.

Idea: Maintain a current **[L,R] matching window** to avoid recomputation.

```
vector<int>z(string s){
int n=s.size(),l=0,r=0;
vector<int>z(n);
for(int i=1;i<n;i++){
if(i<=r)z[i]=min(r-i+1,z[i-l]);
while(i+z[i]<n&& s[z[i]]==s[i+z[i]])z[i]++;
if(i+z[i]-1>r)l=i,r=i+z[i]-1;
}
return z;
}
```

Polynomial Rolling Hash

Complexity: Build $O(N)$, Query $O(1)$

Use: Compare substrings in constant time.

Idea: Store prefix hashes and powers.

```
const int B=31;
vector<int>H,P;
void build(string s){
int n=s.size();
```

```

H.assign(n+1,0);
P.assign(n+1,1);
for(int i=1;i<=n;i++){
P[i]=P[i-1]*B%M;
H[i]=(H[i-1]*B+s[i-1])%M;
}
}

int get(int l,int r){
return (H[r]-H[l]*P[r-l]%M+M)%M;
}

```

Double Hash

Use: Reduce collision probability.

Idea: Compute two hashes using different mod values.

```

mod1=1000000007;
mod2=1000000009;

```

Rabin-Karp

Use: Pattern matching using hashing.

Idea: Compare pattern hash with substring hash.

```

Hash(pattern)==Hash(substring)

```

Manacher

Complexity: $O(N)$

Use: Longest palindromic substring.

Idea: Reuse previously computed palindrome radius.

```

vector<int>manacher(string s){
string t="@";
for(char c:s)t+="#",t+=c;
t+="#$";
int n=t.size();
vector<int>p(n);
for(int i=1,l=1,r=1;i<n-1;i++){

```

```

p[i]=max(0,min(r-i,p[l+r-i]));
while(t[i-p[i]]==t[i+p[i]])p[i]++;
if(i+p[i]>r)l=i-p[i],r=i+p[i];
}
return p;
}

```

LCS

Complexity: $O(NM)$

Use: Longest Common Subsequence.

Idea: DP on prefixes of both strings.

```

for(int i=1;i<=n;i++)
for(int j=1;j<=m;j++)
if(a[i-1]==b[j-1])
dp[i][j]=dp[i-1][j-1]+1;
else
dp[i][j]=max(dp[i-1][j],dp[i][j-1]);

```

Edit Distance

Complexity: $O(NM)$

Use: Minimum insert/delete/replace operations.

Idea: DP on prefixes.

```

if(a[i]==b[j])
dp[i][j]=dp[i-1][j-1];
else
dp[i][j]=1+min({
dp[i-1][j],
dp[i][j-1],
dp[i-1][j-1]
});

```

String Functions

Use: Most common STL string operations.

```

s.substr(l,len);    // substring
s.find(t);          // first occurrence

```

```
s.rfind(t);      // last occurrence
s.erase(pos,len); // erase substring
s.insert(pos,t);  // insert string
s.replace(pos,len,t); // replace substring
s.append(t);      // append string
s.push_back(c);   // append character
s.pop_back();     // remove last char
s.front();        // first character
s.back();         // last character
s.empty();        // empty?
```

Character Functions

Use: Character classification.

```
islower(c); // lowercase?
isupper(c); // uppercase?
isdigit(c); // digit?
isalpha(c); // alphabet?
isalnum(c); // letter or digit?
tolower(c); // lowercase
toupper(c); // uppercase
```

String ↔ Number

Use: Convert between string and numeric types.

```
stoi(s); // string -> int
stoll(s); // string -> long long
stof(s); // string -> float
stod(s); // string -> double
to_string(x); // number -> string
```

String Split

Use: Split by spaces.

```
stringstream ss(s);
string x;
while(ss>>x)v.push_back(x); // tokenize
```

Next Permutation

Use: Generate permutations in lexicographical order.

```
sort(all(s));
do{
}while(next_permutation(all(s))); // next permutation
```

Complexity

Algorithm	Time
KMP	O(N)
Z Algorithm	O(N)
Rolling Hash	Build O(N), Query O(1)
Manacher	O(N)
LCS	O(NM)
Edit Distance	O(NM)

=====

MATHEMATICS & NUMBER THEORY

=====

Binary Exponentiation

Use: Compute $a^b \bmod M$, modular inverse, matrix exponentiation.

Idea: Square the base and multiply only when current bit of exponent is 1.

Time: $O(\log B)$

```
int binexp(int a,int b){
int ans=1;
while(b){
if(b&1)ans=ans*a%M;
a=a*a%M;
b>>=1;
}
return ans;
}
```

Modular Inverse

Use: Modular division (Prime Mod).

Idea: Fermat's theorem $a^{(M-2)}$.

Time: $O(\log M)$

```
int modInv(int a){
return binexp(a,M-2);
}
```

nCr

Use: Combination under modulo.

Idea: Precompute factorial & inverse factorial.

Time: Pre: $O(N)$ Query: $O(1)$

```
vector<int> fact(N + 5, 0), ifact(N + 5, 0);
```

```
int mod(int x, int m = M) { return x % m; }
```

```
int binexp(int a, int b) {
```

```
    int ans = 1ll;
```

```
while (b) {
```

```
    if (b & 1)
```

```
        ans = mod(ans * 1ll * a);
```

```
    a = mod(a * 1ll * a);
```

```
    b >>= 1;
```

```
}
```

```
return mod(ans);
```

```
}
```

```
void pre() {
```

```
    fact[0] = 1ll;
```

```
    for (int i = 1; i <= N; i++)
```

```
        fact[i] = mod(fact[i - 1] * 1ll * i);
```

```
    ifact[N] = binexp(fact[N], M - 2);
```

```
    for (int i = N; i > 0; i--) {
```

```
        ifact[i - 1] = mod(ifact[i] * 1ll * i);
```

```
    }
```

```
}
```

```
int nCr(int n, int r) {
```

```
    if (n < r || r < 0 || n <= 0)
```

```
        return 0ll;
```

```
    if (r == 0 || n == r)
```

```
        return 1ll;
```

```
    return mod(mod(fact[n] * 1ll * ifact[r]) * 1ll * ifact[n - r]);
```

```
}
```

Extended Euclid

Use: Solve $ax+by=\gcd(a,b)$, modular inverse (non-prime).

Idea: Backtrack Euclid recursion.

Time: $O(\log N)$

```
int exgcd(int a,int b,int&x,int&y){
if(!b){x=1;y=0;return a;}
int x1,y1;
int g=exgcd(b,a%b,x1,y1);
x=y1;
y=x1-a/b*y1;
return g;
}
```

Sieve of Eratosthenes

Use: Generate all primes $\leq N$.

Idea: Mark multiples.

Time: $O(N \log \log N)$

```
vector<int>prime;
vector<int>vis(N+1);
void sieve(){
for(int i=2;i<=N;i++){
if(!vis[i]){
prime.push_back(i);
for(int j=i*i;j<=N;j+=i)
vis[j]=1;
}
}
}
```

Smallest Prime Factor (SPF)

Use: Fast prime factorization.

Idea: Store smallest divisor of every number.

Time: Build $O(N \log \log N)$ Query $O(\log N)$

```
vector<int>spf(N+1);
void SPF(){
for(int i=2;i<=N;i++)spf[i]=i;
for(int i=2;i*i<=N;i++){
if(spf[i]==i)
for(int j=i*i;j<=N;j+=i)
if(spf[j]==j)
```

```
spf[j]=i;
}
```

Prime Factorization

Use: Factorize many numbers after SPF.

Idea: Repeatedly divide by smallest prime.

Time: $O(\log N)$

```
while(n>1){
cout<<spf[n]<<" ";
n/=spf[n];
}
```

Euler Totient

Use: Count integers coprime to n .

Idea: Remove every distinct prime factor.

Time: $O(\sqrt{N})$

```
int phi(int n){
int ans=n;
for(int i=2;i*i<=n;i++){
if(n%i==0){
while(n%i==0)n/=i;
ans-=ans/i;
}
}
if(n>1)ans-=ans/n;
return ans;
}
```

Matrix Exponentiation

Use: Fibonacci & linear recurrences.

Idea: Binary exponentiation on matrices.

Time: $O(K^3 \log N)$

```
struct Mat{int a[2][2];};
Mat mul(Mat A,Mat B){
Mat C{};
for(int i=0;i<2;i++)
for(int j=0;j<2;j++)
```

```

for(int k=0;k<2;k++)
C.a[i][j]=(C.a[i][j]+A.a[i][k]*B.a[k][j])%M;
return C;
}
Mat power(Mat A,int n){
Mat R={1,0,0,1};
while(n){
if(n&1)R=mul(R,A);
A=mul(A,A);
n>>=1;
}
return R;
}

```

Fibonacci

Use: Nth Fibonacci.

Idea: Matrix exponentiation.

Time: $O(\log N)$

```

Mat A={1,1,1,0};
cout<<power(A,n).a[0][1];

```

Catalan Number

Use: BST, balanced parentheses, triangulation.

Idea: Closed formula.

Time: $O(1)$ (after nCr)

```

nCr(2*n,n)*binexp(n+1,M-2)%M;

```

Chinese Remainder Theorem (CRT)

Use: Solve multiple congruences $x \equiv a_1 \pmod{m_1}, x \equiv a_2 \pmod{m_2} \dots$

Idea: Merge equations one by one using Extended Euclid.

Time: $O(K \log M)$

```

int exgcd(int a,int b,int &x,int &y){
if(!b){x=1;y=0;return a;}
int x1,y1;

```

```

int g=exgcd(b,a%b,x1,y1);
x=y1;
y=x1-a/b*y1;
return g;
}

```

```

pair<int,int>CRT(int a1,int m1,int a2,int m2){
int x,y;
int g=exgcd(m1,m2,x,y);
if((a2-a1)%g)return{-1,-1};
int l=m1/g*m2;
int t=(__int128)(a2-a1)/g*x%(m2/g);
int ans=(a1+(__int128)m1*t)%l;
if(ans<0)ans+=l;
return{ans,l};
}

```

Example

```

auto [x,lcm]=CRT(a1,m1,a2,m2);

```

x is the smallest non-negative solution, and all solutions are $x + k * lcm$.

Miller Rabin

Use: Fast primality test (64-bit).

Idea: Deterministic witness test.

Time: $O(\log^3 N)$

```

using u64=uint64_t;
using u128=__uint128_t;

u64 mul(u64 a,u64 b,u64 mod){
return (u128)a*b%mod;
}

u64 power(u64 a,u64 d,u64 mod){
u64 r=1;
while(d){
if(d&1)r=mul(r,a,mod);
a=mul(a,a,mod);

```

```

d>>=1;
}
return r;
}

bool check(u64 n,u64 a,u64 d,int s){
u64 x=power(a,d,n);
if(x==1 || x==n-1)return 1;
while(--s){
x=mul(x,x,n);
if(x==n-1)return 1;
}
return 0;
}

bool isPrime(u64 n){
if(n<2)return 0;
for(u64 p:{2,3,5,7,11,13,17,19,23,29,31,37})
if(n%p==0)return n==p;
int s=0;
u64 d=n-1;
while((d&1)==0)d>>=1,s++;
for(u64
a:{2,325,9375,28178,450775,9780504,1795265022
})
if(a%n&& !check(n,a,d,s))return 0;
return 1;
}

```

```

n*(n+1)/2          // Sum 1...N
n*(n+1)*(2*n+1)/6  // Sum of squares
n*(n+1)*(n+2)/6    // Sum of combinations
C(i,2)
2^10≈1e3
2^20≈1e6
2^30≈1e9
2^40≈1e12
2^50≈1e15
2^60≈1e18

```

When to Use What?

Problem	Algorithm
$a^b \bmod M$	Binary Exponentiation
$a/b \bmod M$	Modular Inverse
Combination	nCr
Generate Primes	Sieve
Many Factorizations	SPF
Coprime Count	Totient
Fibonacci	Matrix Expo
Huge Prime Check	Miller Rabin
Multiple Mods	CRT

Formula Box

DYNAMIC PROGRAMMING

Memoization

Use: Top-down DP when states are recursive.

Idea: Save computed states to avoid recomputation.

Time: $O(\text{State})$

```
vector<int>dp(N,-1);
int solve(int n){
    if(n<=1)return 1;
    if(dp[n]!=-1)return dp[n];
    return dp[n]=solve(n-1)+solve(n-2);
}
```

Tabulation

Use: Bottom-up DP.

Idea: Build answer from smaller states.

Time: $O(\text{State})$

```
vector<int>dp(n+1);
dp[0]=1;
for(int i=1;i<=n;i++)
    dp[i]=dp[i-1]+i;
```

0/1 Knapsack

Use: Pick each item at most once.

Idea: Take or skip.

Time: $O(NW)$

```
vector<int>dp(W+1);
for(int i=0;i<n;i++)
    for(int j=W;j>=wt[i];j--)
        dp[j]=max(dp[j],dp[j-wt[i]]+val[i]);
```

Unbounded Knapsack

Use: Unlimited copies of each item.

Idea: Iterate weight forward.

Time: $O(NW)$

```
vector<int>dp(W+1);
for(int i=0;i<n;i++)
    for(int j=wt[i];j<=W;j++)
        dp[j]=max(dp[j],dp[j-wt[i]]+val[i]);
```

Coin Change (Minimum Coins)

Use: Minimum number of coins.

Idea: Relax every coin.

Time: $O(NX)$

```
vector<int>dp(X+1,INF);
dp[0]=0;
for(auto c:coin)
    for(int i=c;i<=X;i++)
        dp[i]=min(dp[i],dp[i-c]+1);
```

Coin Change (Number of Ways)

Use: Count combinations.

Idea: Process each coin once.

Time: $O(NX)$

```
vector<int>dp(X+1);
dp[0]=1;
for(auto c:coin)
    for(int i=c;i<=X;i++)
        dp[i]+=dp[i-c];
```

LIS ($O(N \log N)$)

Use: Longest Increasing Subsequence.

Idea: Maintain smallest possible tail.

Time: $O(N \log N)$

```
vector<int>v;
for(auto x:a){
    auto it=lower_bound(all(v),x);
    if(it==v.end())v.push_back(x);
```

```
else *it=x;
}
cout<<v.size();
```

LCS

Use: Longest Common Subsequence.

Idea: DP on prefixes.

Time: $O(NM)$

```
for(int i=1;i<=n;i++)
for(int j=1;j<=m;j++)
if(a[i-1]==b[j-1])
dp[i][j]=dp[i-1][j-1]+1;
else
dp[i][j]=max(dp[i-1][j],dp[i][j-1]);
```

Edit Distance

Use: Minimum insert/delete/replace.

Idea: DP over prefixes.

Time: $O(NM)$

```
if(a[i]==b[j])
dp[i][j]=dp[i-1][j-1];
else
dp[i][j]=1+min({dp[i-1][j],dp[i][j-1],dp[i-1][j-1]});
```

Subset Sum

Use: Is sum X possible?

Idea: Classic boolean knapsack.

Time: $O(NS)$

```
vector<int>dp(S+1);
dp[0]=1;
for(auto x:a)
for(int j=S;j>=x;j--)
dp[j] |= dp[j-x];
```

Partition Equal Sum

Use: Divide into two equal subsets.

Idea: Subset Sum of total/2.

Time: $O(NS)$

```
int sum=accumulate(all(a),0LL);
if(sum&1)return 0;
```

Then run **Subset Sum(sum/2)**.

Kadane

Use: Maximum subarray sum.

Idea: Extend or restart.

Time: $O(N)$

```
int cur=0,ans=-INF;
for(auto x:a){
cur=max(x,cur+x);
ans=max(ans,cur);
}
```

Prefix DP

Use: DP on prefixes.

Idea: Transition from previous index.

```
dp[0]=0;
for(int i=1;i<=n;i++)
dp[i]=min(dp[i],dp[i-1]+cost);
```

Interval DP

Use: Merge intervals, Matrix Chain.

Idea: Increase interval length.

Time: $O(N^3)$

```
for(int len=2;len<=n;len++){
for(int l=0;l+len<=n;l++){
int r=l+len-1;
for(int k=l;k<r;k++){
dp[l][r]=min(dp[l][r],dp[l][k]+dp[k+1][r]);
}
```

Digit DP

Use: Count numbers satisfying digit constraints.

Idea: Position + Tight + Started + State.

```
int dp[20][2][2];
```

```
int solve(int pos,bool tight,bool start){  
}
```

Bitmask DP

Use: Traveling Salesman, Assignment.

Idea: Mask represents visited states.

```
int dp[1<<20][20];
```

```
int solve(int mask,int last){  
}
```

Tree DP

Use: Independent Set, Matching.

Idea: DP on subtree.

```
void dfs(int u,int p){  
    dp[u][1]=1;  
    for(auto v:g[u]){  
        if(v==p)continue;  
        dfs(v,u);  
        dp[u][0]+=max(dp[v][0],dp[v][1]);  
        dp[u][1]+=dp[v][0];  
    }  
}
```

=====

GREEDY AND SORTING

=====

Binary Search (Last True)

Use: Find maximum answer satisfying a condition.

Idea: Keep last valid answer.

Time: $O(\log N)$

```
int l=0,r=1e9,ans=-1;  
while(l<=r){  
    int m=l+(r-l)/2;  
    if(check(m))ans=m,l=m+1;  
    else r=m-1;  
}
```

Floating Binary Search

Use: Binary search on real values.

Idea: Iterate fixed number of times.

Time: $O(100)$

```
double l=0,r=1e9;  
for(int i=0;i<100;i++){  
    double m=(l+r)/2;  
    if(check(m))r=m;  
    else l=m;  
}
```

Ternary Search

Use: Find maximum/minimum of unimodal function.

Idea: Compare two middle points.

Time: $O(100)$

```
double l=0,r=1e9;  
for(int i=0;i<100;i++){  
    double m1=l+(r-l)/3;  
    double m2=r-(r-l)/3;  
    if(f(m1)<f(m2))l=m1;
```

```
else r=m2;
}
```

Two Pointers

Use: Sorted array, pair sum, unique subarray.

Idea: Move one pointer based on condition.

Time: $O(N)$

```
int l=0,r=n-1;
while(l<r){
if(a[l]+a[r]==x)break;
if(a[l]+a[r]<x)l++;
else r--;
}
```

Sliding Window (Variable Size) ★★ ★★ ★★ ★★

Use: Longest/shortest subarray satisfying a condition.

Idea: Expand r, shrink l until valid.

Time: $O(N)$

```
int l=0,r=0,sum=0,ans=0;
while(r<n){
sum+=a[r++];
while(sum>k){
sum-=a[l++];
}
ans=max(ans,r-l);
}
```

Sliding Window (Count Exactly K)

Use: Number of subarrays with exactly K.

Idea: $\text{Exactly}(K) = \text{AtMost}(K) - \text{AtMost}(K-1)$.

Time: $O(N)$

```
int atMost(int k){
unordered_map<int,int>mp;
int l=0,r=0,res=0;
while(r<n){
```

```
mp[a[r]]++;
while(mp.size()>k){
if(--mp[a[l]]==0)mp.erase(a[l]);
l++;
}
res+=r-l+1;
r++;
}
return res;
}
```

Usage

$\text{atMost}(k) - \text{atMost}(k-1)$;

=====

GEOMETRY

=====

Point Structure

Use: Store 2D points.

Idea: Basic geometry primitive.

```
struct Point{
double x,y;
Point(){}
Point(double x,double y):x(x),y(y){}
Point operator+(Point p){return{x+p.x,y+p.y};}
Point operator-(Point p){return{x-p.x,y-p.y};}
Point operator*(double k){return{x*k,y*k};}
};
```

Dot Product

Use: Angle, projection, perpendicular.

Idea: $a \cdot b = x_1x_2 + y_1y_2$.

Time: $O(1)$

```
double dot(Point a,Point b){
return a.x*b.x+a.y*b.y;
}
```

Cross Product ★★☆☆☆

Use: Orientation, area, convex hull, segment intersection.

Idea: Sign determines left/right turn.

Time: $O(1)$

```
double cross(Point a, Point b){
    return a.x*b.y-a.y*b.x;
}
```

Orientation

Use: Clockwise / Counterclockwise.

Idea: Check sign of cross product.

Time: $O(1)$

```
int orient(Point a, Point b, Point c){
    double v=cross(b-a,c-a);
    if(v>0)return 1;
    if(v<0)return -1;
    return 0;
}
```

Distance

Use: Euclidean distance.

Idea: Pythagoras.

Time: $O(1)$

```
double dist(Point a, Point b){
    return hypot(a.x-b.x,a.y-b.y);
}
```

Distance²

Use: Compare distances without sqrt().

Idea: Faster than Euclidean distance.

Time: $O(1)$

```
double dist2(Point a, Point b){
    double dx=a.x-b.x;
```

```
double dy=a.y-b.y;
return dx*dx+dy*dy;
}
```

Line Equation

Use: Standard line form.

Idea: $Ax+By+C=0$.

```
double A=y2-y1;
double B=x1-x2;
double C=-(A*x1+B*y1);
```

Segment Intersection

Use: Check if two segments intersect.

Idea: Opposite orientations.

Time: $O(1)$

```
bool intersect(Point a, Point b, Point c, Point d){
    int o1=orient(a,b,c);
    int o2=orient(a,b,d);
    int o3=orient(c,d,a);
    int o4=orient(c,d,b);
    return o1!=o2&&o3!=o4;
}
```

Polygon Area

Use: Area of polygon.

Idea: Shoelace Formula.

Time: $O(N)$

```
double area(vector<Point>&p){
    double ans=0;
    int n=p.size();
    for(int i=0;i<n;i++)
        ans+=cross(p[i],p[(i+1)%n]);
    return fabs(ans)/2;
}
```

Convex Hull (Monotonic Chain)

Use: Smallest convex polygon.

Idea: Remove right turns.

Time: $O(N \log N)$

```
sort(all(p));
vector<Point>h;
for(auto x:p){
while(h.size()>1&&cross(h.back()-h[h.size()-2],x-
h.back())<=0)
h.pop_back();
h.push_back(x);
}
```

Rotate Point

Use: Rotate point by angle.

Idea: Rotation matrix.

Time: $O(1)$

```
Point rotate(Point p,double a){
return{
p.x*cos(a)-p.y*sin(a),
p.x*sin(a)+p.y*cos(a)
};
}
```

Point in Rectangle

Use: Bounding box.

Time: $O(1)$

```
x>=x1&& x<=x2&& y>=y1&& y<=y2;
```

Point on Segment

Use: Collinear check.

Time: $O(1)$

```
cross(b-a,p-a)==0&&
dot(p-a,p-b)<=0;
```

Circle

Use: Radius, circumference, area.

Area= $\pi * r * r$

Circumference= $2 * \pi * r$

Triangle Area

Use: Area using coordinates.

Idea: Half of cross product.

Time: $O(1)$

```
fabs(cross(b-a,c-a))/2;
```

Useful Constants

```
const double PI=acos(-1.0);
```

```
const double EPS=1e-9;
```

Floating Comparison

```
fabs(a-b)<EPS; // equal
```

```
fabs(x)<EPS; // zero
```

Geometry Formula Box

Dot>0 -> Acute

Dot=0 -> 90°

Dot<0 -> Obtuse

Cross>0 -> CCW

Cross<0 -> CW

Cross=0 -> Collinear

Triangle Area=|Cross|/2

Polygon Area=Shoelace

Distance²>Distance
(no sqrt)

Complexity

Algorithm	Time
Dot	$O(1)$
Cross	$O(1)$
Orientation	$O(1)$
Distance	$O(1)$
Segment Intersection	$O(1)$
Polygon Area	$O(N)$
Convex Hull	$O(N \log N)$